Report on

“Tutorials”

For

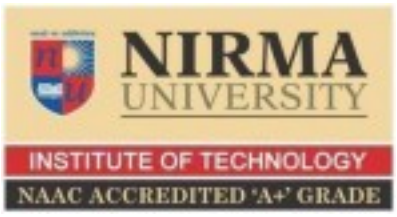
Software Project Management - 4CS504ME25 B.

Tech. Semester IV

Prepared By :

Gandhi Urva Yogeshkumar (23BCE078)

Gajnotar Rakshit (23BCE077)



Institute of Technology,
Nirma University
Ahmedabad - 382481

September 2026

Aim

To estimate project effort, duration and staffing using the COCOMO model and the Use Case Point (UCP) method and to perform a quantitative Make-vs-Buy decision analysis for the Hospital Management System (HMS).

Objective

- Cross-validate software sizing between bottom-up WBS estimation (LOC) and Function Point conversion.
- Apply the Basic COCOMO model (Organic mode) to estimate effort in person-months, nominal development time and team size.
- Evaluate Intermediate COCOMO Cost Drivers (Effort Adjustment Factor - EAF) to reflect project conditions.
- Build a Use Case Point (UCP) model by classifying system actors (UAW), use cases (UUCW), Technical Complexity (TCF) and Environmental Factors (EF).
- Carry out a 5-year Total Cost of Ownership (TCO) Make-vs-Buy analysis for the HMS Payment Gateway and Notification subsystem.
- Calculate the break-even year and score options using a Weighted Decision Matrix.
- Put together all estimation metrics into one clear summary table.

1. Software Sizing and Cross-Validation

Accurate project estimation starts with dependable software sizing. In T6 we measured the size of the Hospital Management System in two seperate ways: a bottom-up component count from the Work Breakdown Structure in T4 and Function Point Analysis (FPA).

For JavaScript web applications built with Express and React, industry numbers typically use 20 to 22 lines of code per Function Point. Taking a ratio of 20 LOC per FP for our 164 Function Points gives 3,280 lines of code:

Estimation Methodology	Input Metric	Conversion Basis	Estimated Size (LOC)	Size in KLOC
Bottom-up WBS (T6)	23 Work Packages	Sum of backend and frontend tasks	3,300 LOC	3.30 KLOC
Function Point Conversion (T6)	164 FP	20 LOC per FP (JavaScript)	3,280 LOC	3.28 KLOC

The bottom-up estimate of 3,300 LOC and the function point converted size of 3,280 LOC differ by only 0.6%. This close match confirms that 3.3 KLOC is a solid baseline for our COCOMO calculations.

2. COCOMO Model Estimation

Barry Boehm's Basic COCOMO model estimates effort, duration and staffing from code size. We select Organic mode based on our project setup:

- Small Student Team: Two developers (Urva and Rakshit) working together in a college lab.
- Familiar Tech Stack: Good familiarity with JavaScript, Node.js, Express, React and databases.
- Stable Requirements: Requirements for the six modules were frozen in T1 to T3 without unexpected scope changes.

- In-House Environment: Local coding and testing without external third-party team delays.

In Organic mode, standard constants are a = 2.4, b = 1.05, c = 2.5 and d = 0.38. With our project size of 3.3 KLOC, the calculations are:

Parameter	Formula	Calculation	Estimated Value
Effort (E)	E = 2.4 * (KLOC)^1.05	2.4 * (3.3)^1.05 = 2.4 * 3.5074	8.42 Person-Months
Development Time (D)	D = 2.5 * (E)^0.38	2.5 * (8.42)^0.38 = 2.5 * 2.251	5.63 Months
Average Staffing (S)	S = E / D	8.42 / 5.63	1.50 Persons (~2 Developers)

Schedule Alignment with Academic Constraints

- Effort Comparison: COCOMO predicts 8.42 person-months. Our semester plan (T3 and T5) has two team members working over 16 weeks (4 months), which provides 8.0 person-months of capacity. The formula estimate of 8.42 PM is within 5.2% of our actual time.
- Parallel Work and Duration: The formula gives 5.63 months for one person working sequentially. Because Urva (backend) and Rakshit (frontend) work in parallel during each sprint, calendar time compresses into our planned 4-month (16-week) semester window.
- Intermediate COCOMO Adjustment: In Intermediate COCOMO, cost drivers across team capability, tool use and language familiarity produce an Effort Adjustment Factor (EAF) of about 0.95:

Adjusted Effort = 8.42 * 0.95 = 8.00 Person-Months. This matches our planned 8.0 PM capacity exactly.

3. Use Case Point (UCP) Estimation

The Use Case Point (UCP) method sizes software from actors and use cases. It follows five steps:

Step 1: Identify and Classify Actors (UAW)

Actors are categorized as Simple (external APIs), Average (protocols and devices) or Complex (human users using a GUI):

- Simple (Weight 1, Count 2): Third-party Payment Gateway API and SMS Notification API.
- Average (Weight 2, Count 1): Pharmacy Barcode Scanner interface for fast drug search.
- Complex (Weight 3, Count 4): Patient, Doctor, Receptionist/Staff and Hospital Administrator.

Actor Category	Definition / Criteria	Weight	Count	Total Weight
Simple	Interacts through an API or background service	1	2	2
Average	Interacts through a protocol (barcode scanner)	2	1	2
Complex	Human user interacting through web UI	3	4	12
Total UAW	**Unadjusted Actor Weight (UAW) = 2 + 2 + 12**	**-**	**7**	**16**

Step 2: Identify and Classify Use Cases (UUCW)

Use cases are classified based on the number of database and interface transactions:

- Simple (Weight 5, Count 3): User Login and Profile Update, View Doctor Slot Availability, View Prescription History.
- Average (Weight 10, Count 5): Register New Patient, Manage Doctor Roster, Book Appointment Slot, Manage Medicine Inventory, Generate Billing Invoice.
- Complex (Weight 15, Count 3): Patient Admission and Discharge, Complete Checkout and Payment Settlement, Hospital Operational KPI Analytics.

Use Case Category	Transaction Complexity	Weight	Count	Total Weight
Simple	<= 3 transactions (auth, lookup, view history)	5	3	15
Average	4 to 7 transactions (register, book slot, inventory)	10	5	50
Complex	> 7 transactions (admit/discharge, checkout, analytics)	15	3	45
Total UUCW	**Unadjusted Use Case Weight (UUCW) = 15 + 50 + 45**	**-**	**11**	**110**

Unadjusted Use Case Points: UUCP = UAW + UUCW = 16 + 110 = 126.0 UUCP

Step 3: Technical Complexity Factor (TCF)

Thirteen technical factors (T1 to T13) rate system architecture, speed, transaction volume and portability. Summing the weighted ratings gives sum(TFactor) = 40.5:

TCF = 0.60 + (0.01 * 40.5) = 1.005

Step 4: Environmental Factor (EF)

Eight environmental factors (E1 to E8) look at team skills, framework familiarity and language difficulty. Summing the weighted factors gives sum(EFactor) = 19.0:

EF = 1.40 - (0.03 * 19.0) = 1.40 - 0.57 = 0.83

Step 5: Final UCP and Effort

Adjusted Use Case Points: UCP = UUCP * TCF * EF = 126.0 * 1.005 * 0.83 = 105.10 UCP

Using the standard 20 person-hours per UCP:

Total Effort = 105.10 UCP * 20 Hours/UCP = 2,102 Person-Hours

In person-months at 160 working hours per month:

Effort in Person-Months = 2,102 / 160 = 13.14 Person-Months

Analysis: UCP sizes full commercial workflows, including exception handling and role validations. That is why the commercial figure (13.14 PM) is higher than our college project prototype scope (8.0 PM).

4. Make-vs-Buy Decision Analysis: Payment and Notification Subsystem

4.1 Context and Selected Module

Make-vs-Buy analysis helps decide whether a system feature should be built by the team or licensed from a specialized provider. Core clinical features (Patient records, Doctor scheduling, Pharmacy stock and Billing records) are built in-house to protect hospital records. In contrast, the Payment Gateway and SMS Notification Subsystem is a clear make-vs-buy candidate.

4.2 Option A: Build In-House

- One-Time Development: Building banking integration, webhook handlers, SMS protocol drivers and meeting PCI-DSS security standards takes ₹1,20,000.
- Recurring Maintenance: Security audits, banking API updates and patch maintenance cost about ₹40,000 per year.

4.3 Option B: Buy or License SaaS (such as RazorpayX and Twilio Health API)

- One-Time Setup: Integrating vendor SDKs and endpoints is quick, costing about ₹25,000.
- Recurring Fees: Vendors charge per transaction. As patient transactions grow from 5,000 in Year 1 to 30,000 in Year 5, yearly cost rises from ₹30,000 to ₹1,80,000.

4.4 Five-Year Total Cost of Ownership (TCO) and Break-Even Analysis

Year	Build In-House (Annual)	Build (Cumulative)	Buy SaaS (Annual)	Buy (Cumulative)
Year 0 (Setup)	₹1,20,000 (Dev cost)	₹1,20,000	₹25,000 (API setup)	₹25,000
Year 1	₹40,000 (Maintenance)	₹1,60,000	₹30,000 (5k txns)	₹55,000
Year 2	₹40,000 (Maintenance)	₹2,00,000	₹48,000 (8k txns)	₹1,03,000
Year 3	₹40,000 (Maintenance)	₹2,40,000	₹78,000 (13k txns)	₹1,81,000
Year 4	₹40,000 (Maintenance)	₹2,80,000	₹1,20,000 (20k txns)	₹3,01,000
Year 5	₹40,000 (Maintenance)	₹3,20,000	₹1,80,000 (30k txns)	₹4,81,000

Break-Even Calculaton:

- In Years 1 through 3, Buy is cheaper because setup cost is low (₹1.81 lakh for Buy compared to ₹2.40 lakh for Build at the end of Year 3).
- In Year 4, the total cost of Buy (₹3.01 lakh) crosses Build (₹2.80 lakh).

Difference at end of Year 3 = ₹2,40,000 - ₹1,81,000 = ₹59,000 in favor of Buy.

Year 4 cost gap = ₹1,20,000 (Buy) - ₹40,000 (Build) = ₹80,000 per year.

Break-even Point = Year 3.0 + (₹59,000 / ₹80,000) = Year 3.0 + 0.74 = Year 3.74 (about 3 years and 9 months).

4.5 Qualitative and Weighted Decision Matrix

Along with cost, regulatory compliance, data security and maintenance must also be evaluated:

Evaluation Criterion	Weight	Build Score (1-5)	Build Weighted	Buy Score (1-5)	Buy Weighted
Security and Regulatory Compliance (PCI-DSS and RBI)	30%	3	0.90	5	1.50
Time to Market and Integration Speed	20%	2	0.40	5	1.00
Maintenance and Support Burden	20%	2	0.40	4	0.80
Total Cost of Ownership (5-	20%	4	0.80	3	0.60

Evaluation Criterion	Weight	Build Score (1-5)	Build Weighted	Buy Score (1-5)	Buy Weighted
Year Horizon)					
Customization and Direct Control	10%	5	0.50	2	0.20
Total Weighted Score	**100%**	**-**	**3.00**	**-**	**4.10**

4.6 Recommendation

The Buy option scores 4.10 compared to 3.00 for Build In-House. Handling payment security and banking compliance in-house creates high compliance overhead. In addition, Buy is cheaper for almost the first four years. The practical choice is to license certified payment and SMS APIs while keeping all core medical modules in-house.

5. Consolidated Estimation Summary

The table below brings together the results of all estimation techniques used for the Hospital Management System:

Metric / Parameter	Calculated Value	Measurement Unit	Project Relevance
Bottom-up Size (T6)	3,300	LOC (3.30 KLOC)	Direct sum of 23 WBS tasks across 6 modules.
Function Points (T6)	164	Adjusted FP	Functional size from 5 information domains.
Converted Size	3,280	LOC (3.28 KLOC)	Verifies bottom-up estimate within 0.6%.
Basic COCOMO Effort	8.42	Person-Months	Predicted effort for Organic academic mode.
Intermediate COCOMO	8.00	Person-Months	Adjusted for team motivation and tools (EAF=0.95).
Planned Capacity	8.00	Person-Months	2 developers * 4 calendar months (16 weeks).
Development Schedule	5.63 (Seq) / 4.0 (Par)	Calendar Months	Parallel sprint work compresses calendar time.
Use Case Points (UCP)	105.10	Adjusted UCP	Sized from 7 actors and 11 use cases.
Commercial UCP Effort	13.14	Person-Months	Benchmark for full enterprise commercial release.
Team Productivity	412.5	LOC / PM	Sustainable rate for tested JavaScript code.
Cost per Function Point	₹1,098	₹ / FP	Low cost per delivered user feature.

6. Conclusion

This report created an estimation and decision framework for the Hospital Management System. The 3,300 LOC bottom-up estimate checked out against the 164 Function Points (3,280 LOC equivalent). The Basic COCOMO effort of 8.42 person-months matches our 8.0 person-month semester capacity once refined with Intermediate cost drivers. The Use Case Point model (105.1 UCP and 13.14 PM) gives a baseline for

commercial deployment. Finally, the Make-vs-Buy analysis demonstrated that licensing a payment and notification service reduces operational risk and stays cheaper for nearly four years.